

המדריך הבא נועד לעזור לך להבין את פרויקט השחמט שלך לעומק, כדי שתוכל להציג אותו בביטחון מלא בהגנה [cite: 8, 9]. בעל-פה. הוא מסביר בשפה פשוטה את המושגים המרכזיים ומראה היכן הם מופיעים בקוד

מדריך למידה והכנה להגנת הפרויקט: משחק שחמט מאובטח

מבט-על: איך כל החלקים מתחברים. 1.

[cite: 78]. הפרויקט הוא משחק שחמט שמשחק בין שני מחשבים דרך הרשת

(Client) "שיושבת במרכז ומנהלת את הכול, ויש תוכניות "לקוח (Server) "יש תוכנית אחת שנקראת "שרת" אחת לכל שחקן - שמציגות את הלוח ושולחות מהלכים [cite: 79]

- [cite: 81]. השרת מחכה לחיבורים בכתובת 127.0.0.1 (המחשב המקומי) ובפורט 5555
- [cite: 82]. כל שחקן מתחבר לשרת, נרשם או מתחבר למערכת
- השרת מחזיק את הלוח האמיתי, בודק כל מהלך שהלקוח שולח (כדי למנוע רמאויות), ומשדר את המצב [cite: 85, 86]. החדש לשני השחקנים. הלקוח רק מצייר את הלוח על המסך ושולח בקשות

2. (Synchronization) וסנכרון (Threads) נימות

Thread מה זה בעצם

תהליך הוא תוכנית שרצה עם (Process) צריך קודם להבין מה זה תהליך, (Thread) כדי להבין מה זה נימה. זיכרון נפרד משלה. נימה היא "נתיב ריצה" בתוך התוכנית הזו. כל הנימות באותה תוכנית חולקות את אותו זיכרון [cite: 96, 97, 98, 99]

בפרויקט Threads למה צריך

תאר לך מלצר אחד במסעדה שחייב לסיים לטפל בשולחן אחד עד הסוף לפני שהוא ניגש לשולחן הבא. אם לקוח אחד מתלבט חצי שעה, כל שאר הלקוחות יחכו וייתבישו [cite: 91, 92]. כך גם בשרת: אם השרת היה מטפל רק בלקוח אחד בכל רגע, שחקן שחושב על המהלך שלו היה תוקע את כל שאר השחקנים. הפתרון הוא לפתוח נימה נפרדת לכל לקוח. זה כמו לתת מלצר אישי לכל שולחן - כולם מטופלים במקביל. [cite: 93, 94]

?איך זה מומש בקוד

ClientHandler בלב השרת, יש לולאה שמחכה לחיבורים. ברגע שלקוח מתחבר, השרת פותח עובד אובייקט Runnable שמממש את הממשק (Thread-עוטף אותו ב, [cite: 100, 102, 103] start(): נדש וקורא ל Thread-עוטף אותו ב, Runnable שמממש את הממשק)

```
Socket socket = server.accept();
ClientHandler handler = new ClientHandler(socket, auth, matchmaker);
```

```
new Thread(handler).start();
```

הבעיה הנסתר: מירוץ על נתונים (Race Condition)

מכיוון שכל הנימות חולקות את אותו זיכרון, מה יקרה אם שני שחקנים ינסו לשנות את הלוח בדיוק באותה אלפית [cite: 111, 112]. שנייה? הנתונים עלולים להתנגש ולהיהרס

אנחנו, synchronized-כשאנחנו מסמנים מתודה כ. **synchronized** הוא המילה השמורה Java-הפתרון ב". רק נימה אחת יכולה להיכנס למתודה הזו בכל רגע נתון. שאר הנימות חייבות לחכות בתור": Java-אומרים ל Matchmaker כדי לבצע מהלך בטוח) ובמחלקת Game נעשה בזה שימוש במחלקת [cite: 113] (שחקנים לתור [cite: 115, 116, 117].

3. (OOP) תכנות מונחה עצמים

[cite: 406]: בצורה קלאסית OOP-הפרויקט בנוי מ-19 מחלקות שמיישמות את עקרונות ה

היכן בפרויקט	הסבר	עיקרון
מוגדרת Piece מחלקת ויש לה מתודה, abstract-כ שכל כלי getMoves מופשטת [cite: 199, 203]. חייב לממש בדרכו	יצירת מחלקת אב המגדירה "מה" צריך לעשות מבלי להגדיר "איך". אי אפשר לייצר אובייקט ישירות ממחלקה מופשטת [cite: 199, 217]	הפשטה (Abstraction)
מחלקות הכלים השונים (כמו Piece-יורשות מ Knight, Rook) את הצבע, המיקום ופעולות תנועה בסיסיות [cite: 199, 206]	מחלקה המקבלת תכונות ופעולות (ממחלקת אב) (חוסך קוד כפול [cite: 199, 220])	ירושה (Inheritance)
כשהלוח רוצה לדעת לאן כלי יכול לזוז, הוא פשוט קורא הוא לא ().getMoves ל-piece- צריך לדעת איזה כלי זה - כל כלי יחשב את התנועה המיוחדת שלו [cite: 210, 211]. בעצמו	רב-צורתיות: אותה פקודה מתנהגת אחרת בהתאם לסוג האובייקט שמריץ אותה [cite: 199, 211]	פולימורפיזם (Polymorphism)

4. SHA-256 + Salt) אבטחה וערבול סיסמאות

?למה לא שומרים סיסמה רגילה

אם היינו שומרים את הסיסמאות כפי שהן בקובץ הטקסט (טקסט גלוי), כל מי שהיה משיג את הקובץ היה יודע [cite: 167, 168]. (Hash) את כל הסיסמאות. במקום זאת, אנחנו מעבירים את הסיסמה דרך פונקציית ערבול ערבול הוא חד-כיווני (בניגוד להצפנה שהיא דו-כיוונית). קל מאוד לחשב את תוצאת הערבול מהסיסמה, אבל אי [cite: 169, 170, 184]. SHA-256 אפשר להרכיב את הסיסמה בחזרה מתוך תוצאת הערבול. הפרויקט משתמש באלגוריתם

?Salt מה זה

הבעיה היא שאם שני משתמשים יבחרו את הסיסמה "1234", שניהם יקבלו בדיוק את אותו ערבול, והדבר יחשוף מחרוזת אקראית וייחודית לכל משתמש - לפני - Salt מידע להאקרים. כדי למנוע זאת, מוסיפים לכל סיסמה [cite: 172, 174, 175]. שמערבלים. כך סיסמאות זהות מקבלות תוצאות שונות לחלוטין ותוצאת הערבול. כשהמשתמש מנסה להתחבר, השרת Salt-שומרים רק ה (users.json) בקובץ המשתמשים השמור, מערבל הכל ובודק אם התוצאה תואמת לזו שבקובץ Salt-לוקח את הסיסמה שהוזנה, מוסיף לה את ה [cite: 177, 181, 182, 186]

5. JSON-תקשורת רשת ו

[cite: 140]. (קצה הצינור שדרכו שולחים נתונים) Sockets תקשורת המחשבים נעשית באמצעות

- UDP. כי הוא מבטיח שכל הודעה תגיע ליעדה ובסדר הנכון TCP-השתמשנו ב **UDP ולא TCP למה** [cite: 142, 143, 144, 145] אמנם מהיר יותר, אבל הודעות עלולות ללכת לאיבוד - ובשחמט, איבוד של מהלך אחד יהרוס את כל המשחק
- השרת והלקוח מדברים ביניהם בשפה מוסכמת שמבוססת על פורמט טקסטואלי: **JSON פרוטוקול** (move, login), שאומר לשרת מה סוג ההודעה (למשל type לכל הודעה יש שדה JSON שנקרא והשרת, switch. מנתב את ההודעה בהתאם בעזרת משפט [cite: 227, 228, 229, 238]

6. (שאלות שכיחות מהבוחנים) ותשובות מנצחות

1. ש: מי אחראי לבדוק שהמהלך חוקי, השרת או הלקוח?
ת: השרת! השרת הוא הסמכות. אם הלקוח היה בודק, האקר היה יכול לשנות את קוד הלקוח ולשלוף [cite: 86, 369, 370]. לשרת מהלכים לא חוקיים כדי לרמות. השרת מחזיק את לוח האמת ובודק כל מהלך
2. ש: איך המסך מתעדכן בלי להיתקע כשהוא מחכה להודעה מהשרת?
של (Timer) ת: הלקוח משתמש בנימת רקע שמקשיבה לשרת ושמה הודעות נכנסות בתור. במקביל, שעון דוגם את התור כל 100 מילי-שניות ומעדכן את הציור על המסך בצורה (Swing) ממשק המשתמש

[cite: 119, 312, 313] בטוחה

3. מחליט איזה מהלך לעשות? (AI-ש: איך המחשב ה)

היא אוספת את כל המהלכים החוקיים, (Greedy) "ת: הבינה המלאכותית פועלת בשיטה "חמדנית מנקדת כל אחד מהם (נותנת משקל גדול לאכילת כלים של היריב ובנוסף קטן על התקרבות למרכז הלוח), [cite: 260, 261, 262, 263, 266] ובחרת את המהלך עם הציון הגבוה ביותר

טיפ זהב להגנה: הבוחן לא מצפה שתזכור קוד בעל-פה. הוא רוצה לראות שאתה מבין למה בחרת בגישה תמיד תתחיל בהסבר על הבעיה שניסית לפתור. (TCP או למה בחרת Thread מסוימת (למשל, למה בחרת [cite: 74, 75, 76] בהצלחה